

PROJECT REPORT ON DEVELOPER'S DIARY

“A Full-Stack Web Application for Personal Developer Journaling”

A full stack project Report submitted for the Partial Fulfilment of Project Requirements

Submitted By:
Khemraj Nepal
Ramkot, Kathmandu, Nepal
Repository:
github.com/KhemJava/developer-s-diary

Date of Release: 31 August 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	1
LIST OF ABBREVIATIONS	2
LIST OF FIGURES	3
LIST OF TABLES	4
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Objectives	1
1.3 Scope	1
CHAPTER 2: SYSTEM ANALYSIS	2
2.1 Problem Definition	2
2.2 Functional Requirements	2
2.3 Non-Functional Requirements	2
CHAPTER 3: PROJECT METHODOLOGY	3
CHAPTER 4: KEY FEATURES	4
4.1 Authentication	4
4.2 Diary Management	4
4.3 Security	4
4.4 User Experience	4
CHAPTER 5: SYSTEM DESIGN	5
5.1 System Architecture	5
5.2 Data Flow Diagram – Level 0 (Context Diagram)	5
5.3 Data Flow Diagram – Level 1	5
5.4 Entity Relationship (ER) Diagram	6
5.5 Database Schema Summary	7
CHAPTER 6: IMPLEMENTATION	9
6.1 Redesigned “Add Diary Entry” Interface	9
6.2 Email Verification on Registration	9
6.3 Key Modules Delivered	9
CHAPTER 7: API AND BACKEND DESIGN	10
7.1 API Endpoints	10
7.2 Authentication Flow	10
7.3 Security & Authorization	11
CHAPTER 8: TESTING	13
CHAPTER 9: LIMITATIONS AND FUTURE OUTCOMES	15
CHAPTER 10: CONCLUSION	16
REFERENCES	17
APPENDICES	18

LIST OF ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish-based Cryptographic Password Hashing Function
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity Relationship
FR	Functional Requirement
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JAR	Java Archive
JWT	Java JSON Web Token (library)
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
NFR	Non-Functional Requirement
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
TC	Test Case
UUID	Universally Unique Identifier

LIST OF FIGURES

Figure 5.1: System Architecture

Figure 5.2: Context-level Data Flow Diagram

Figure 5.3: Level-1 Data Flow Diagram

Figure 5.4: Entity Relationship Diagram

Figure 8.1: Authentication Flow

Figure 10.1: Home Page

Figure 10.2: Dashboard

Figure 10.3: Redesigned Add Diary Entry Form

Figure 10.4: Login – Unverified Account with Resend Option

Figure 10.5: Email Verification Confirmation

LIST OF TABLES

Table 2.1: Functional Requirements

Table 2.2: Non-Functional Requirements

Table 3.1: Project Methodology Phases

Table 5.1: Database Schema Summary

Table 6.1: Technology Stack

Table 7.1: API Endpoints

Table 8.1: Test Cases and Results

CHAPTER 1: INTRODUCTION

1.1 Background

Developer's Diary is a full stack journaling web application developed especially for developers and students in order to note down the issues they encounter, what they have learned from those experiences, and the tasks they need to perform the next day. Keeping this kind of log is one of the best practices among software engineers to track progress and avoid making mistakes again and again.

1.2 Objectives

- To design and implement a secure authentication system using JSON Web Tokens (JWT).
- To allow each user to create, read, update, delete, and search their own diary entries.
- To verify the user's email address is validated before granting login access to avoid any false registration.
- To have a visually appealing interface to add diary entries.

1.3 Scope

The system encompasses the user registration process along with email verification, authentication, as well as CRUD operations on user diary. It doesn't include password recovery functionality, social log-in options, nor any form of moderation since they fall under future work scope.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Problem Definition

The developers often fail to remember about their previous solutions to certain problems. The traditional book will not allow you to look for information; besides, it won't be available on other computers or devices and will not be password protected.

2.2 Functional Requirements

Table 2.1: Functional Requirements

ID	Requirement
FR-1	User can register with username, email and password.
FR-2	System sends a verification email; the account stays inactive until the link is clicked.
FR-3	User can log in and receive a signed JWT on success.
FR-4	Login is rejected with a clear message if the email is not yet verified, with an option to resend the link.
FR-5	Authenticated user can add, view, edit, delete and search diary entries.
FR-6	A user can only access their own diary entries, never another user's.

2.3 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Category	Requirement
Security	Passwords hashed with BCrypt; stateless JWT auth; verification tokens expire after 24 hours.
Usability	Responsive layout; animated, icon-guided form for adding entries; clear inline error/success feedback.
Performance	REST endpoints respond within acceptable time for a single-instance PostgreSQL-backed deployment.
Portability	Backend packaged as a standard Spring Boot JAR; frontend as a standard Create-React-App build.

CHAPTER 3: PROJECT METHODOLOGY

This project followed an iterative development approach that is common with software development projects in academia and on a personal level. The phases in the development were sequential such that the features discussed in this report (redesigned entry form and email validation) were part of the next iteration in the same code base.

Table 3.1: Project Methodology Phases

Phase	Activities
1. Requirement Analysis	Identified the core problem (developers need a private, searchable log of daily technical work) and defined functional/non-functional requirements (Section 2).
2. System Design	Designed the three-tier architecture, ER model, and data flow between client, server, and database (Section 5).
3. Database Design	Modelled the <code>app_user</code> , <code>verification_token</code> , and <code>dairy_post</code> tables and their relationships; chose PostgreSQL with Hibernate's schema auto-update for iterative development.
4. Backend Development	Implemented REST controllers, services, and JPA repositories in Spring Boot; added the <code>VerificationToken</code> entity and <code>EmailService</code> for the new verification flow.
5. Authentication Implementation	Implemented BCrypt password hashing, Spring Security's filter chain, and stateless JWT issuance/validation; extended <code>CustomUserDetailsService</code> to gate login on the account's enabled flag.
6. Frontend Development	Built the React SPA with route-based components; redesigned <code>AddPost.jsx</code> with animated SVG graphics and built the new <code>VerifyEmail.jsx</code> page.
7. Integration	Connected the React client to the REST API via the <code>api.js</code> service layer, including token storage, auth headers, and 401/403 handling.
8. Testing	Manually verified each user flow end-to-end against the test cases in Section 9.

CHAPTER 4: KEY FEATURES

4.1 Authentication

- User registration with username, email and password.
- Email verification is required before an account can log in.
- Stateless authentication using JSON Web Tokens (JWT).
- Passwords hashed with BCrypt – never stored in plain text.
- Login / logout with client-side session (token) management.

4.2 Diary Management

- Create new diary entries (problem faced, what was learned, improvements, tomorrow's plan, notes).
- View all entries on a personal dashboard.
- Edit existing entries.
- Delete entries.
- Search entries by keyword.

4.3 Security

- User-specific data access – entries are scoped server-side to the owning account.
- Accounts remain disabled until the registered email address is verified.
- Verification tokens are single-use and automatically expire after 24 hours.
- Server-side ownership validation on every diary read/write, independent of client input.

4.4 User Experience

- Responsive interface that adapts to different screen sizes.
- Animated diary-entry form with custom SVG graphics and icon-guided fields.
- Live character counters on each field.
- Visual completion indicators and a progress bar as the form is filled.
- Success animations (checkmark and confetti) on completing key actions.

CHAPTER 5: SYSTEM DESIGN

5.1 System Architecture

The application follows a three-tier architecture: a React single-page application (client tier), a Spring Boot REST API secured with Spring Security and JWT (application tier), and a PostgreSQL database with an external SMTP service for email delivery (data/external tier).

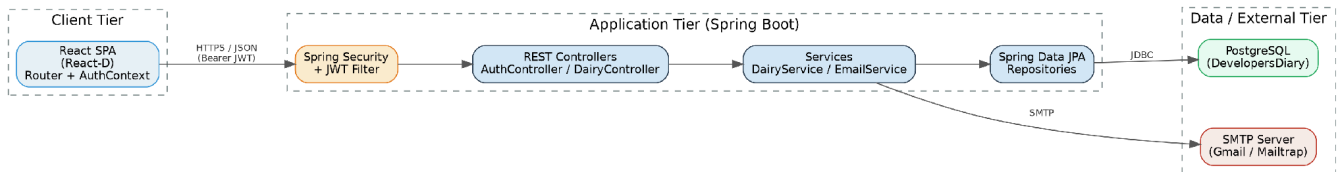


Figure 5.1: System Architecture

5.2 Data Flow Diagram – Level 0 (Context Diagram)

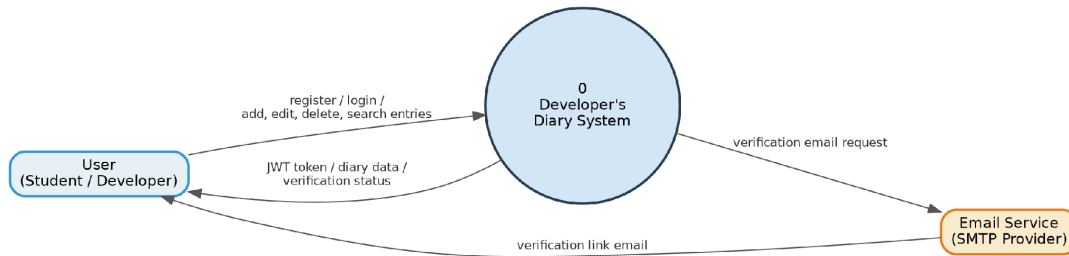


Figure 5.2: Context-level Data Flow Diagram

5.3 Data Flow Diagram – Level 1

The level-1 DFD decomposes the system into its four core processes: registration, email verification, login, and diary management.

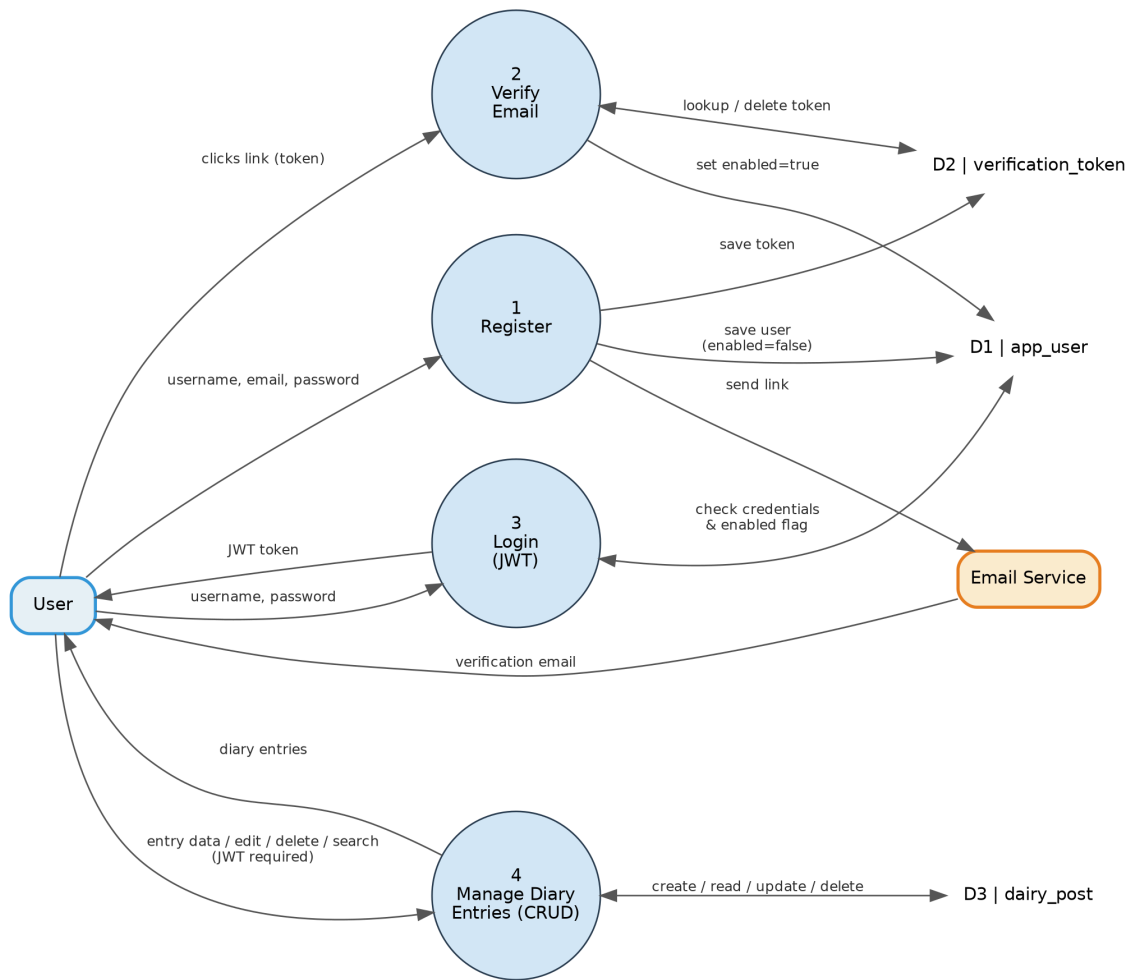


Figure 5.3: Level-1 Data Flow Diagram

5.4 Entity Relationship (ER) Diagram

Three entities back the system. A User can issue at most one active VerificationToken and can author many DairyPost records; each diary entry is tagged with its owner's username at creation time and is never exposed to other users.

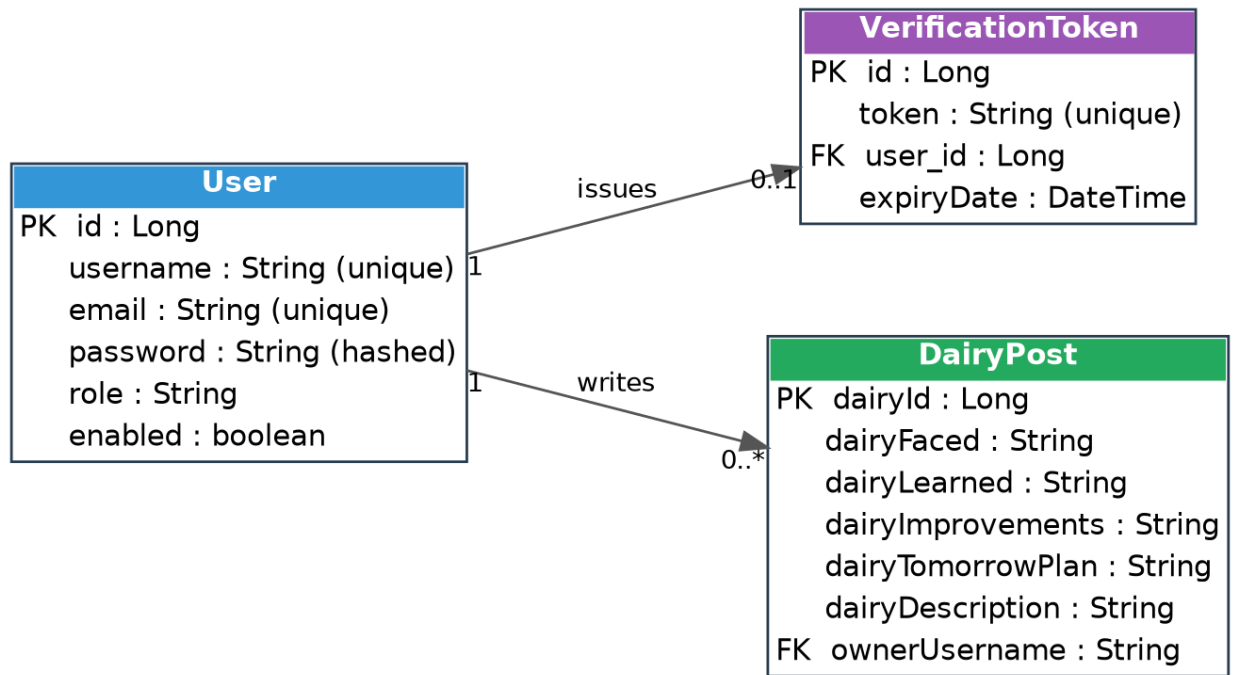


Figure 5.4: Entity Relationship Diagram

5.5 Database Schema Summary

Table 5.5: Database Schema Summary

Table	Key Columns	Purpose
app_user	id (PK), username, email, password, role, enabled	Stores account credentials and verification status.
verification_token	id (PK), token, user_id (FK), expiryDate	Tracks one-time email-verification links.
dairy_post (DairyPost)	dairyId (PK), dairyFaced, dairyLearned, dairyImprovements, dairyTomorrowPlan, dairyDescription, ownerUsername	Stores each diary entry, scoped to its owner.

5.6 Technology Stack

Table 5.6: Technology Stack

Layer	Technology
Frontend	React 18, React Router v6, plain CSS3 with custom animations and inline SVG graphics
Backend	Java 21, Spring Boot 3.4, Spring Security, Spring Data JPA, Spring Mail
Authentication	JSON Web Tokens (JWT 0.12) with BCrypt password hashing
Database	PostgreSQL
Build Tools	Maven (backend), npm / react-scripts (frontend)

CHAPTER 6: IMPLEMENTATION

6.1 Redesigned “Add Diary Entry” Interface

The Add Diary Entry screen was redesigned using a gradient hero banner and an animated inline SVG illustration that consists of a floating notebook, a wiggling pen, and shining sparks. Each form element is displayed in the form of its own card, which has an accompanying custom icon, live character counter, and a checkmark once it is filled out. The top part has a progress bar that gets filled in when the user fills in each of the five sections. Upon successfully submitting the form, an animated checkmark (using SVG stroke animation technique) and confetti explode before going back to the dashboard. No UI library has been used in this case.

6.2 Email Verification on Registration

Initially, a new account was activated straightaway. The new implementation creates an account in the disabled state and produces a one-time and time-bound (24 hours) token which is sent to the email entered during the registration process. Spring Security’s authentication manager first looks at the status (enabled/disabled) of the account and then the password; thus, an unconfirmed account is not authenticated with an appropriate error message, and not a vague error message, when logging in. A one-click “resend verification email” option becomes available in the frontend for that situation.

6.3 Key Modules Delivered

- AuthController – register, login, verify, and resend-verification endpoints.
- VerificationToken entity and repository, with automatic 24-hour expiry.
- EmailService – sends the verification email via Spring Mail / SMTP.
- Updated CustomUserDetailsService – ties Spring Security's account-enabled check to email verification.
- Redesigned AddPost.jsx and companion styles – animated, icon-driven entry form.
- New VerifyEmail.jsx page and updated Login/Register components.

CHAPTER 7: API AND BACKEND DESIGN

7.1 API Endpoints

All authentication endpoints are namespaced under `/api/auth` and are open to unauthenticated clients; all diary endpoints require a valid JWT and act only on the calling user's own data, resolved server-side from the token via `Authentication.getName()`.

Table 7.1: API Endpoints

Method	Endpoint	Auth Required	Description
POST	<code>/api/auth/register</code>	No	Create a new account (starts disabled) and send a verification email.
POST	<code>/api/auth/login</code>	No	Authenticate and return a signed JWT; rejects unverified or invalid accounts.
GET	<code>/api/auth/verify?token=</code>	No	Confirm a verification token and enable the account.
POST	<code>/api/auth/resend-verification</code>	No	Reissue a verification token/email for an unverified account.
GET	<code>/dairyPosts</code>	Yes	Return all diary entries owned by the logged-in user.
GET	<code>/dairyPost/{dairyId}</code>	Yes	Return a single entry, only if owned by the caller (else 404).
GET	<code>/dairyPosts/keyword/{keyword}</code>	Yes	Search the caller's own entries by keyword.
POST	<code>/addPost</code>	Yes	Create a new entry, saved under the caller's username.
PUT	<code>/dairyPost</code>	Yes	Update an entry, only if owned by the caller (else 404).
DELETE	<code>/dairyPost/{dairyId}</code>	Yes	Delete an entry, only if owned by the caller (else 404).
GET	<code>/load</code>	Yes	Load sample/default entries for the logged-in user.

7.2 Authentication Flow

Login is a two-stage check – account status, then credentials – followed by stateless JWT-based access to every protected endpoint, as shown below.

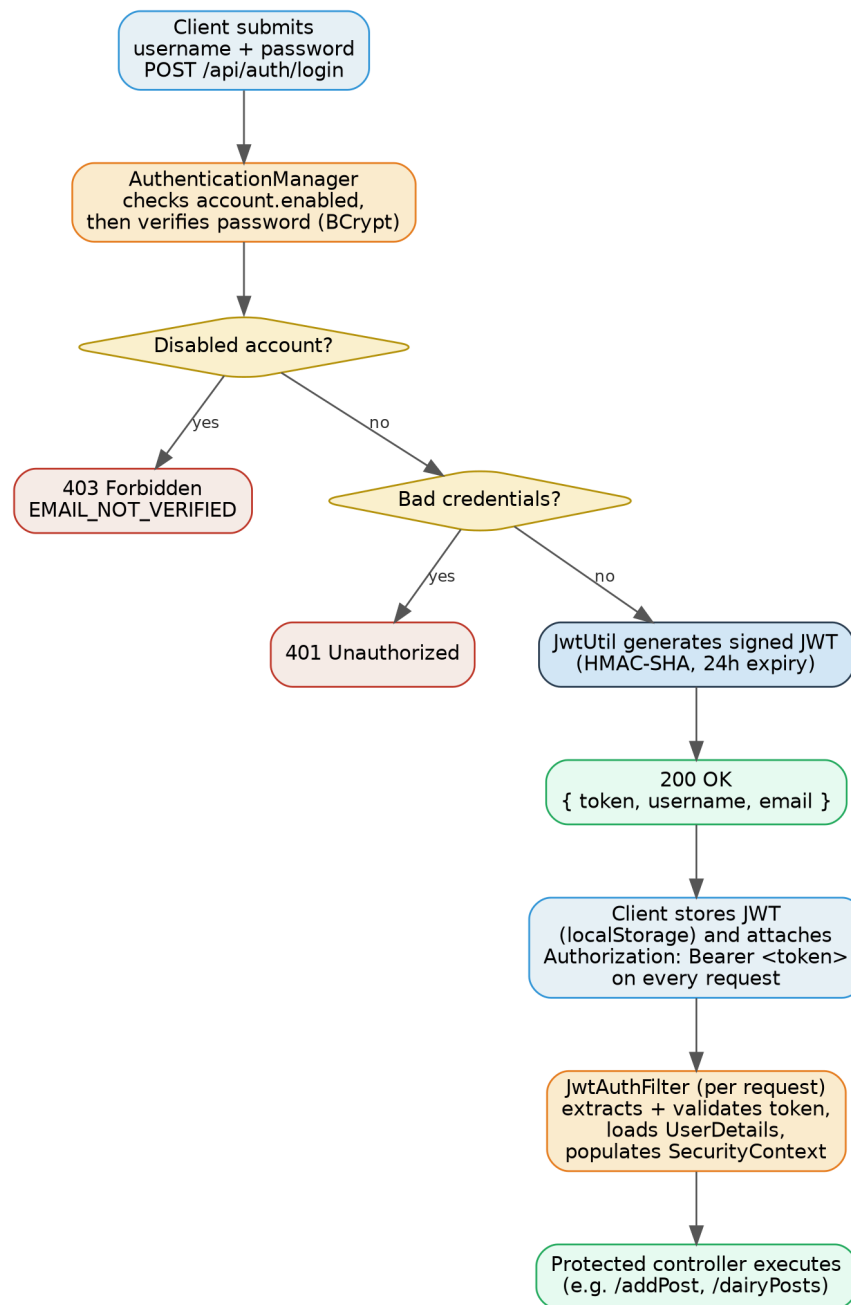


Figure 8.1: Authentication Flow

7.3 Security & Authorization

- No passwords are ever stored or compared in plain text – BCrypt hash function is used on password storage and DaoAuthenticationProvider of Spring Security will check it at login.
- Authentication is stateless: no session on the server side is maintained; authentication happens for each request by using the JWT in the Authorization: Bearer header, which is checked by a custom OncePerRequestFilter (JwtAuthFilter).
- Tokens are signed by using HMAC-SHA key (jwt.secret) and their expiration time is 24 hours (jwt.expiration).

- The 'enabled' attribute of an account is ensured by Spring Security pre-authentication checks; therefore, an unconfirmed account will be refused even before checking the password.
- Verifications are single-use, random (UUID) and have 24-hour expiration period from issuance; resend-verification doesn't give away the existence of any email address in the system.
- Every request for a diary entry read/write resolves the owner of the diary by using the authenticated principal, not by client input; for the entries owned by other users, a response 404 is returned.
- CORS is limited to the known front-end origin (<http://localhost:3000>).

CHAPTER 8: TESTING

These verification and diary management flows were manually verified against the following test scenarios before the delivery.

Table 8.1: Test Cases and Results

Test ID	Test Case	Input / Steps	Expected Result	Actual Result	Status
TC-01	Register new user	Submit valid username, email and password on the Register form	Account created with enabled = false; verification email sent	As expected	Pass
TC-02	Duplicate registration	Register again with an already-used username or email	HTTP 409 Conflict with a specific “already taken/registered” message	As expected	Pass
TC-03	Login before verification	Log in immediately after registering, before clicking the email link	HTTP 403 with “Please verify your email” message	As expected	Pass
TC-04	Resend verification email	Click “Resend email” on the login page with a registered, unverified email	New token issued; confirmation email re-sent	As expected	Pass
TC-05	Verify with valid token	Click the verification link received by email	Account set to enabled = true; token deleted; success message shown	As expected	Pass
TC-06	Verify with expired token	Click a verification link older than 24 hours	HTTP 410 Gone; page offers to resend a new link	As expected	Pass
TC-07	Login after verification	Log in with the now-verified account	HTTP 200 with a valid JWT returned	As expected	Pass
TC-08	Add diary entry	Fill the required fields (Faced, Learned) and submit	Entry saved under the logged-in user; success animation shown; redirected to dashboard	As expected	Pass

Test ID	Test Case	Input / Steps	Expected Result	Actual Result	Status
TC-09	Add entry – missing required field	Submit the form with “Problem Faced” left blank	Inline validation error; request not sent to the server	As expected	Pass
TC-10	Cross-user data isolation	Authenticated user attempts to fetch/edit another user's entry ID	Request rejected – entries are scoped server-side to ownerUsername	As expected	Pass
TC-11	Expired/invalid JWT	Call a protected endpoint with an expired or tampered token	HTTP 401 Unauthorized; client clears session and redirects to login	As expected	Pass
TC-12	Search diary entries	Enter a keyword that matches an existing entry	Matching entries returned and displayed on the dashboard	As expected	Pass

CHAPTER 9: LIMITATIONS AND FUTURE OUTCOMES

- No password-reset (forgot password) flow is currently implemented.
- No pagination on the dashboard; performance may degrade with very large entry counts.
- Mail credentials are stored in `application.properties`; a production deployment should move these to environment variables or a secrets manager.
- Future work could add tagging/categorisation of entries, a calendar view, and export to PDF/Markdown.

CHAPTER 10: CONCLUSION

The presented project provides developers with a secure diary application on the stack level. Additionally, this version of the project contains a considerably improved interface for adding entries and an email verification process. The project meets all of the functional requirements and ensures per-user data isolation according to the standards of three-tier web architecture.

REFERENCES

References are listed in IEEE style, in order of citation.

- [1] VMware, Inc., “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/>. [Accessed: 31-Aug-2026].
- [2] VMware, Inc., “Spring Security Reference Documentation,” Spring.io. [Online]. Available: <https://docs.spring.io/spring-security/>. [Accessed: 31-Aug-2026].
- [3] Meta Platforms, Inc., “React Documentation,” React.dev. [Online]. Available: <https://react.dev/>. [Accessed: 31-Aug-2026].
- [4] J. Hanson, “JJWT: Java JWT – JSON Web Token for Java and Android,” GitHub. [Online]. Available: <https://github.com/jwt/jjwt>. [Accessed: 31-Aug-2026].
- [5] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL.org. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: 31-Aug-2026].
- [6] K. Java, “Developer's Diary [Source Code Repository],” GitHub. [Online]. Available: <https://github.com/KhemJava/developer-s-diary>. [Accessed: 31-Aug-2026].

APPENDICES

The Home page and the Dashboard were not affected by this iteration and are depicted below as captured in the actual running application. The other three pages have been re-captured to show the features discussed in this report.

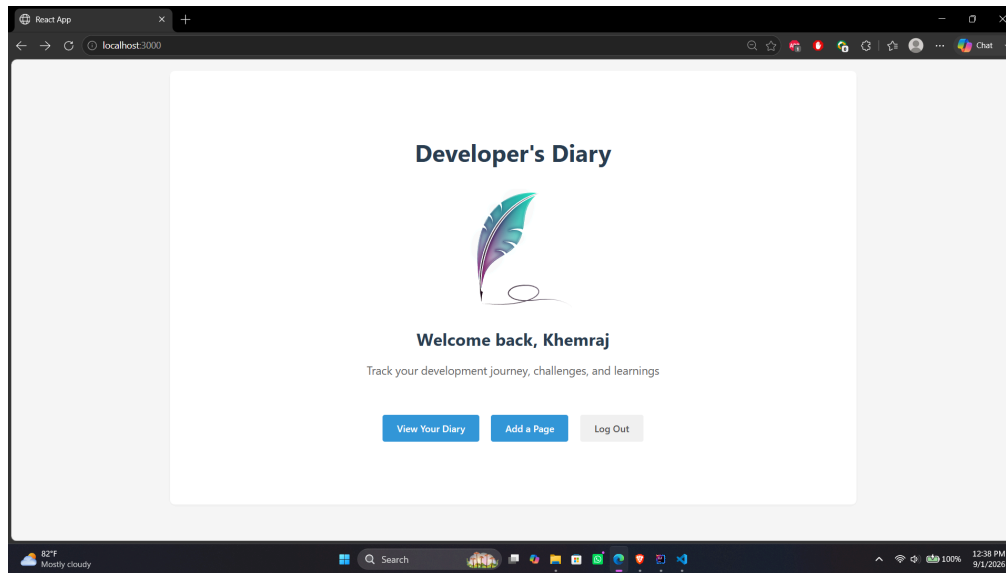


Figure 10.1: Home Page

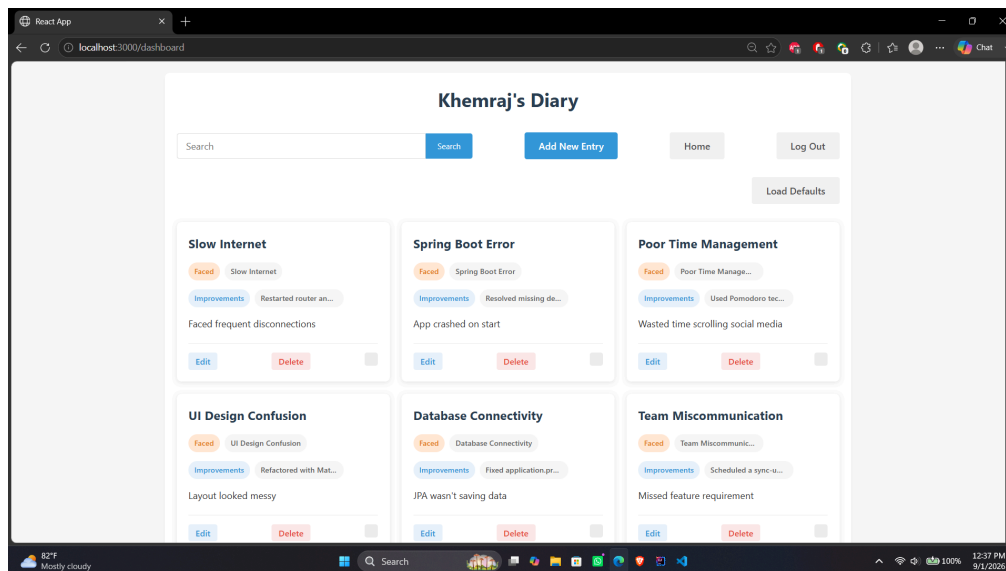


Figure 10.2: Dashboard



Capture Today's Journey

A few honest notes now save you hours of context-switching later.

3 / 5 sections filled


Problem Faced *

64 characters


What You Learned *

108 characters


Areas for Improvement

92 characters


Tomorrow's Plan

0 characters


Additional Notes

0 characters

+ Add Entry

Cancel

[View Your Diary](#)

Figure 10.3: Redesigned Add Diary Entry Form

React App

localhost:3000/register

Create an Account

Username*

Email*

Password*

Confirm Password*

Create Account

Cancel

Already have an account? [Log in](#)

52°F

Mostly cloudy

Search

12:38 PM

9/1/2025

Figure 10.4: Registration

The login form is titled "Log In" in a large, bold, dark blue font. Below the title is a red-bordered box with the text "Please verify your email before logging in." in red. Underneath this is a yellow-bordered box containing the text "Didn't get the email?" and a text input field with the email "bishal@example.com". To the right of the input field is a grey button labeled "Resend email". Below the yellow box are two input fields: "Username*" with the value "bishal_dev" and "Password*" with masked characters ".....". At the bottom are two buttons: a blue "Log In" button and a grey "Cancel" button.

Figure 10.5: Login – Unverified Account with Resend Option

The email verification confirmation screen has a title "Email Verification" in a large, bold, dark blue font. Below the title is a light green box with a green checkmark icon and the text "Email verified! You can now log in." in green. At the bottom is a blue button labeled "Go to Login".

Figure 10.6: Email Verification Confirmation

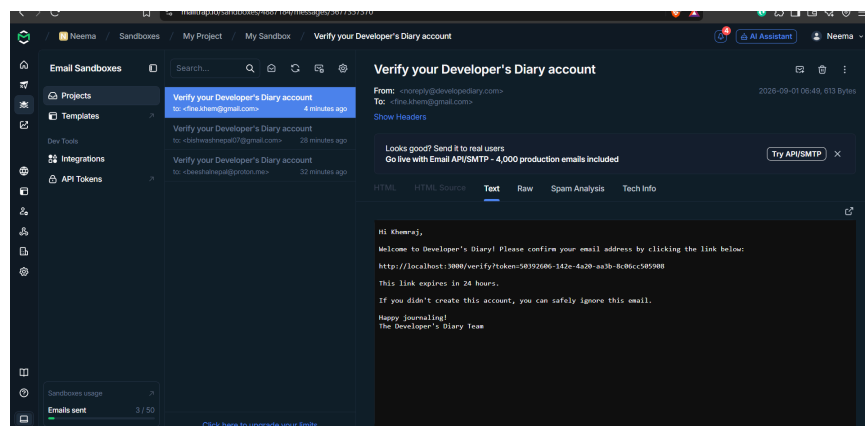


Figure 10.7: SMTP Validation using Mailtrap